Capstone Project Assessment

Project 2 – Build and Secure the Auth0-Backed AI Application

Overview

This project builds and secures an AI chat application using Auth0 as the identity provider. It uses OAuth 2.0 Authorization Code flow with PKCE, scope-based access control, TOTP multi-factor authentication, attack protection, short-lived machine-to-machine (M2M) credentials, post-login Actions and Single Sign-On (SSO). A Node.js (Express) API acts as the resource server and validates RS256-signed JWTs issued by Auth0.

1. Auth0 Tenant Setup

1.1 Creating Applications

I created a free Auth0 developer tenant and registered two applications:

- **SecureNova AI Chat** is a Regular Web Application for the AI chat front end. It uses the Authorization Code grant (with PKCE) and the Refresh Token grant.
- **SecureNova AI Agent** is a Machine-to-Machine application for the AI agent. It uses the Client Credentials grant and is authorized to call the AI Chat API.

1.2 Configuring URLs Required

For the Regular Web Application I configured:

- **Allowed Callback URLs:** http://localhost:3000/callback, http://localhost:3001/callback
- **Allowed Logout URLs:** http://localhost:3000, http://localhost:3001
- **Allowed Web Origins / Allowed Origins (CORS):** http://localhost:3000, http://localhost:3001

The M2M application does not use browser redirects, so it needs no callback URLs or CORS origins. It is restricted to the Client Credentials grant and to the scopes I explicitly authorized.

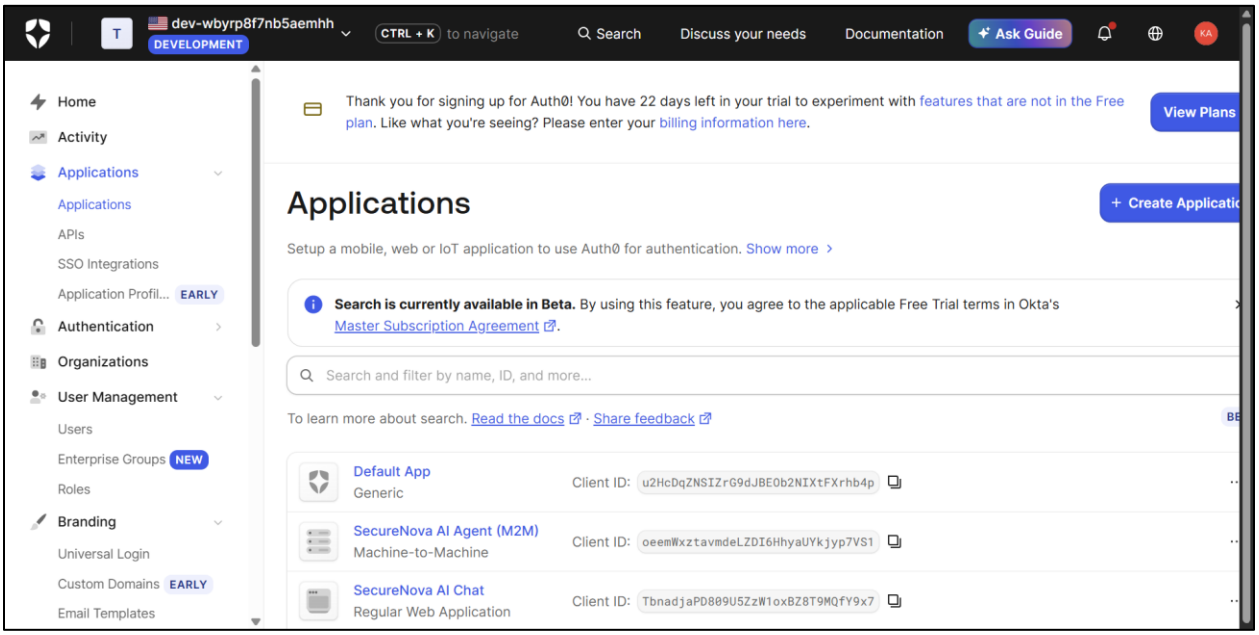


Figure 1: Auth0 Dashboard, Applications page showing the Regular Web Application and the Machine-to-Machine application registered and visible.

2. Implementation of Authorization Code (OAuth 2.0) with PKCE Flow

2.1 API and Custom Scopes

I registered an API in Auth0 with the identifier **https://securenova-api** and the RS256 signing algorithm. I defined two custom scopes:

- **`read:ai-data`** is a user-level scope that allows access to the `/chat` endpoint.
- **`write:admin`** is an admin-level scope that allows access to the `/admin` endpoint.

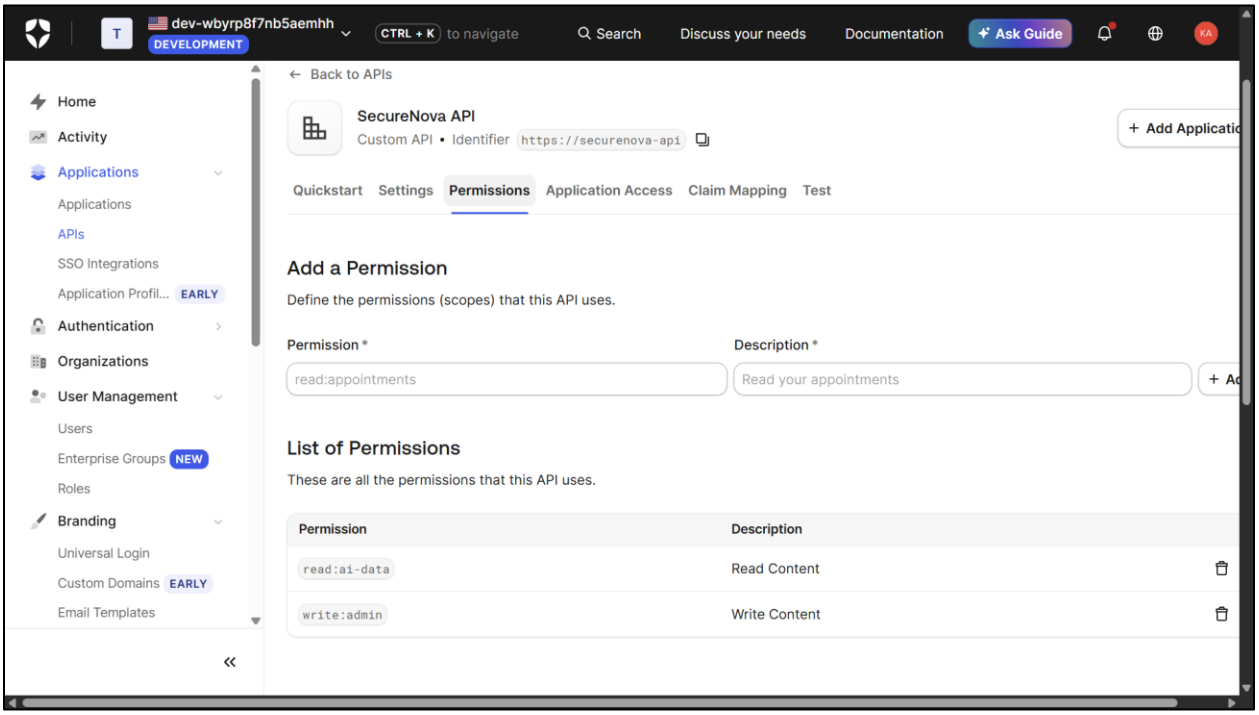


Figure 2: Auth0 APIs page showing the registered SecureNova API with the two custom scopes, `read:ai-data` and `write:admin`.

2.2 Authorization Code + PKCE flow

The web application implements the Authorization Code flow with PKCE:

1. The app generates a random `code_verifier` and derives a `code_challenge` using SHA-256 (`code_challenge_method=S256`).
2. The user is redirected to Auth0's `/authorize` endpoint with the `code_challenge`, the requested audience and scope `openid profile email read:ai-data`.
3. After login, Auth0 returns an authorization code to the callback URL.
4. The app exchanges the code plus the original `code_verifier` at `/oauth/token` for an access token and ID token.

PKCE ensures that an intercepted authorization code cannot be redeemed without the original verifier.

2.3 Scope enforcement (403 on the admin endpoint)

The API validates the token signature, issuer and audience, and enforces scopes per route: `/chat` requires `read:ai-data` and `/admin` requires `write:admin`. I called both endpoints with the user-level token (which only holds `read:ai-data`):

- `GET /chat` returned **200 OK**.
- `GET /admin` returned **403 Forbidden** with `insufficient_scope`.

This demonstrates that a user-level token is rejected by the admin-scoped endpoint.

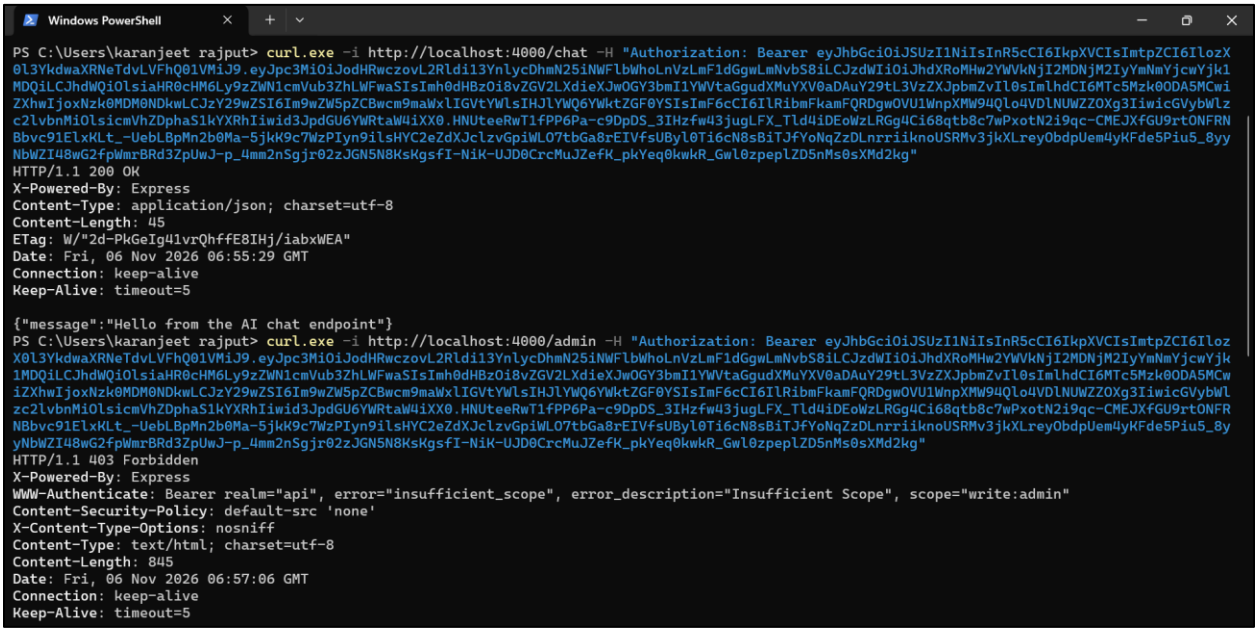


Figure 3: Terminal output of `curl` calls with the user-level token: `/chat` returns 200 OK and `/admin` returns 403 Forbidden (insufficient_scope).

2.4 JWT verification at jwt.io

I decoded the access token at jwt.io and verified:

- **Algorithm:** RS256 in the header.
- **`iss`:** https://<tenant>.auth0.com/, the Auth0 tenant that issued the token.
- **aud:** https://securenova-api, the API the token is intended for.
- **exp:** the expiry timestamp.
- **`scope`:** includes `read:ai-data` but not `write:admin`.

- **`agent_id`:** the custom claim added by the post-login Action (See in Section 5).

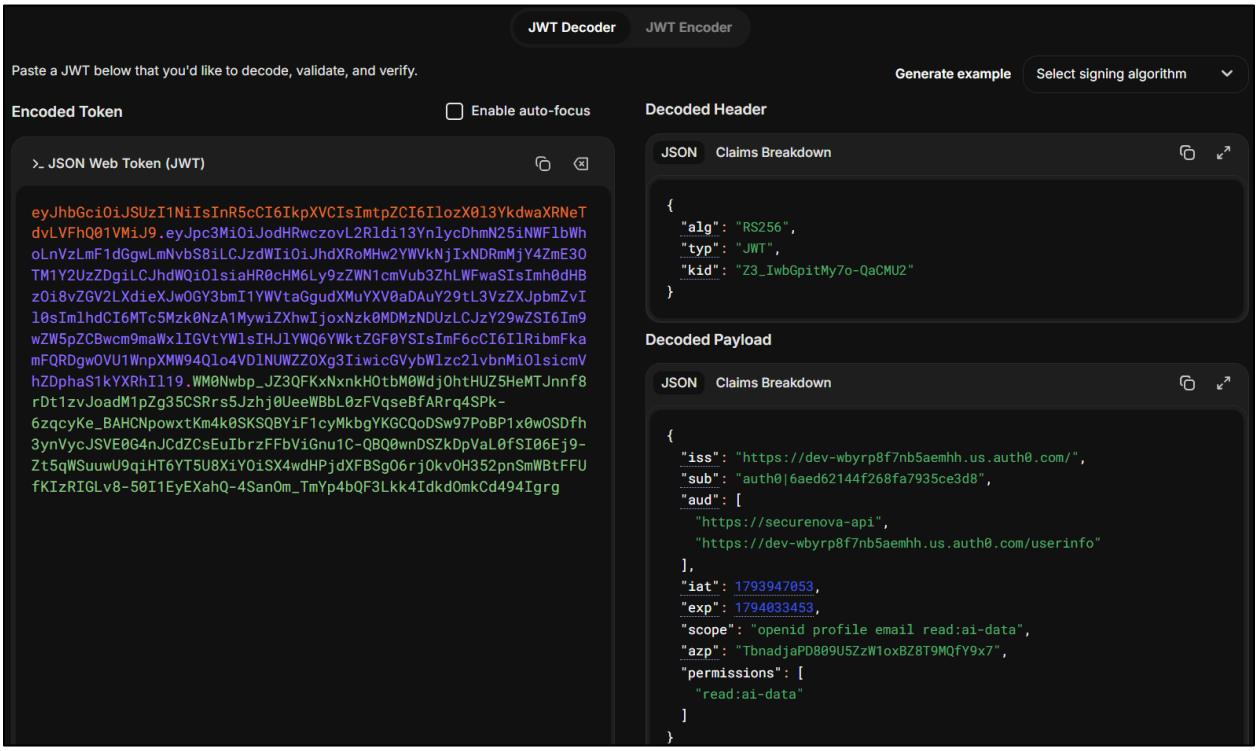


Figure 4: jwt.io decoded access token showing RS256 algorithm and the aud, `iss`, exp, scope and `agent_id` claims.

3. MFA and Attack Protection

3.1 Branding (Universal Login)

I customized Auth0 Universal Login with a custom logo and brand colours so the login experience matches the application.

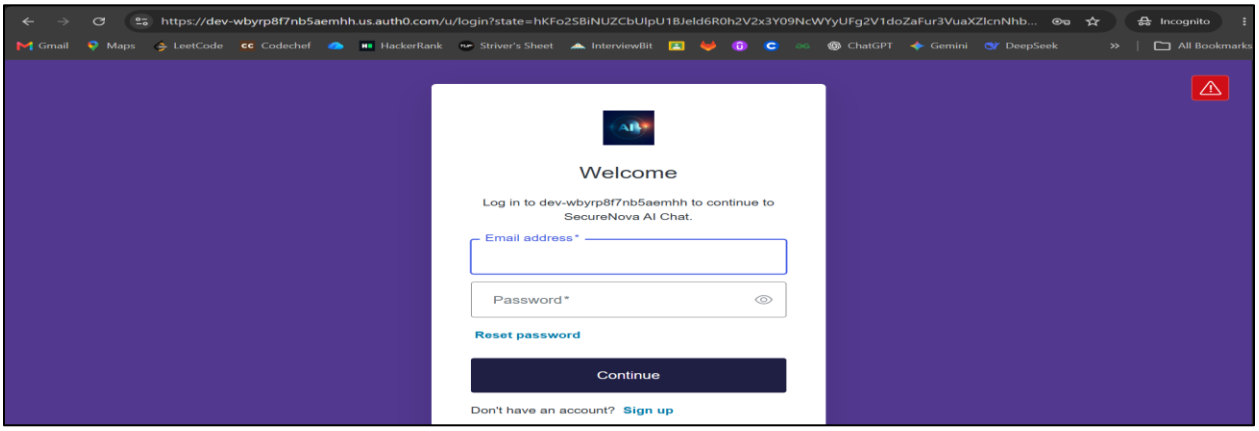


Figure 5: Auth0 Universal Login page in the browser with custom branding applied.

3.2 TOTP-based MFA

I enabled **One-time Password (TOTP)** under Security → Multi-factor Auth, disabled the other factors, and set the policy to **Always**, so MFA is required for all users. On the next login, the user is prompted to scan a QR code with an authenticator app and then enter the 6-digit code. On later logins, the user is challenged for the TOTP code.

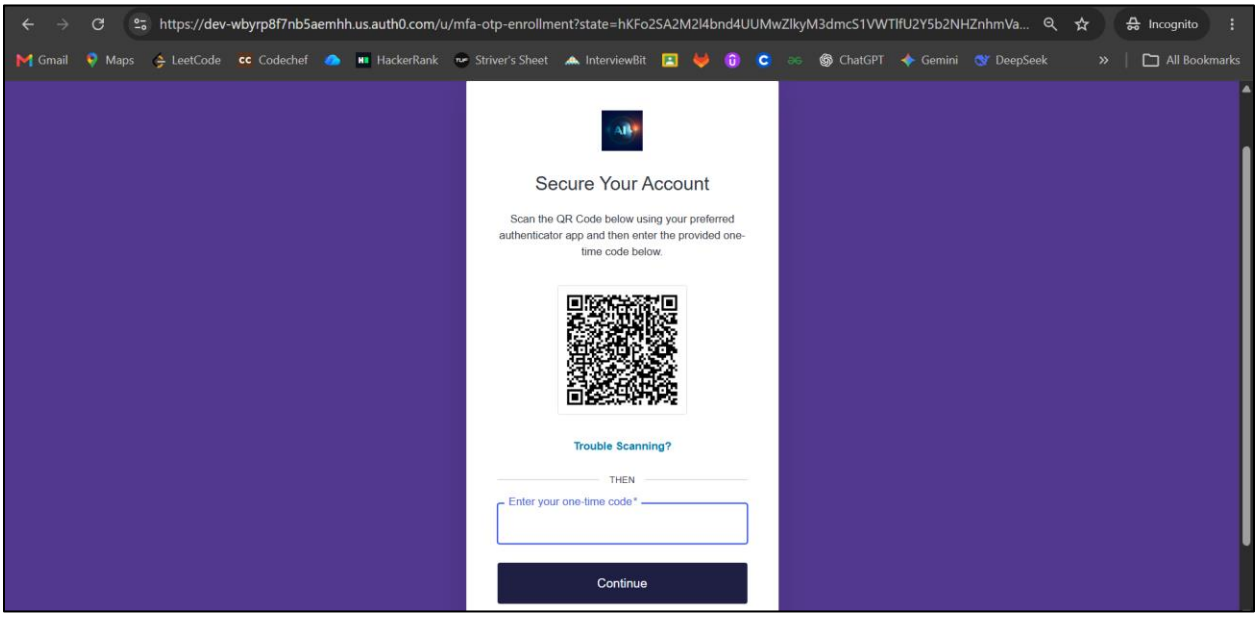


Figure 6: Auth0 MFA enrolment screen / TOTP challenge displayed in the browser.

3.3 Brute-Force Protection and Suspicious IP Throttling

Under Security → Attack Protection I configured:

- **Brute-Force Protection:** enabled, with a maximum of **10** failed attempts and the shields set to *Block* and *Notify*.
- **Suspicious IP Throttling:** enabled, to slow down and block high-rate login or signup attempts from a single IP address.

(Note on the 15-minute lockout: the Auth0 dashboard does not expose a configurable 15-minute timer for Brute-Force Protection. A blocked user is unblocked through the unblock link in the notification email or by an administrator.)

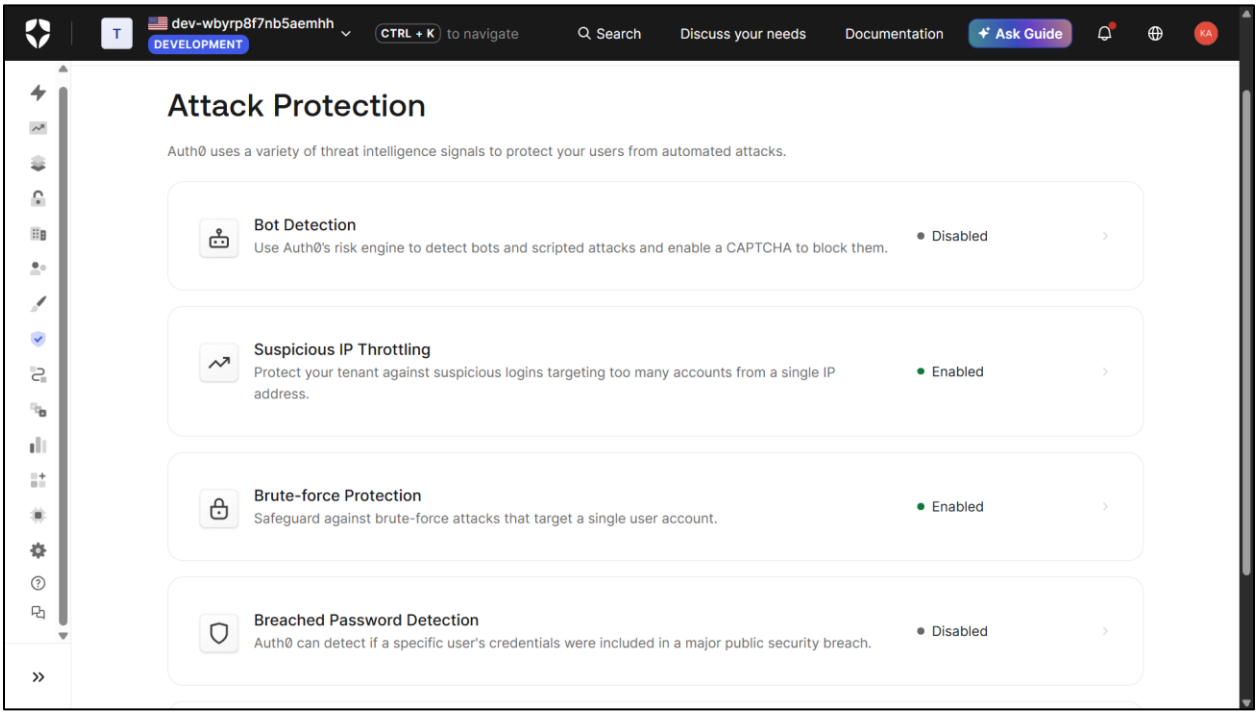


Figure 7: Auth0 Attack Protection panel showing Brute-Force Protection (max 10 attempts) and Suspicious IP Throttling both enabled.

4. Credential Rotation

I wrote a script (`rotate.js`) that demonstrates short-lived credentials. The API's token lifetime was set to **60 seconds**. The script:

1. Requests an M2M access token from Auth0 using the Client Credentials grant (`expires_in = 60`).
2. Calls the `/chat` endpoint with the refresh token and records the response (**200 OK**).
3. Waits for the token to expire (about 70 seconds).
4. Replays the same expired token against `/chat` and records the response (**401 Unauthorized**).

Every step is printed with an ISO timestamp as evidence. This shows that stolen or leaked M2M tokens become useless after 60 seconds, and that the client must request a new token to continue, which is the rotation behaviour.

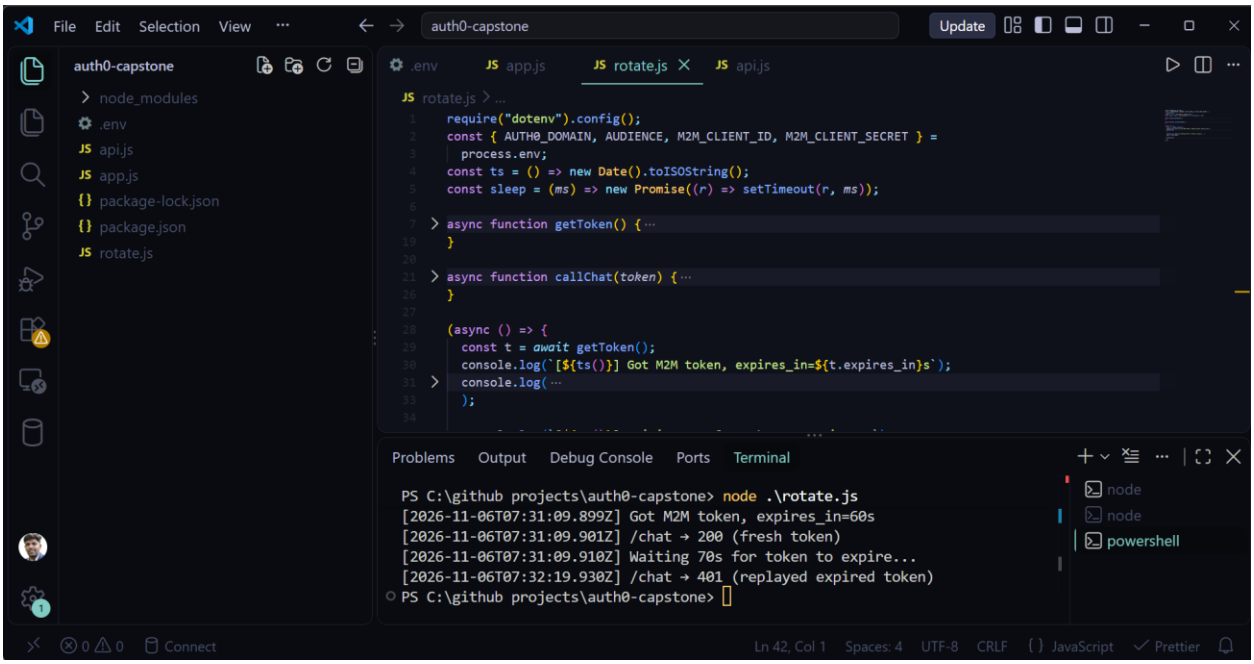


Figure 8: Terminal output of the credential rotation script with timestamps.

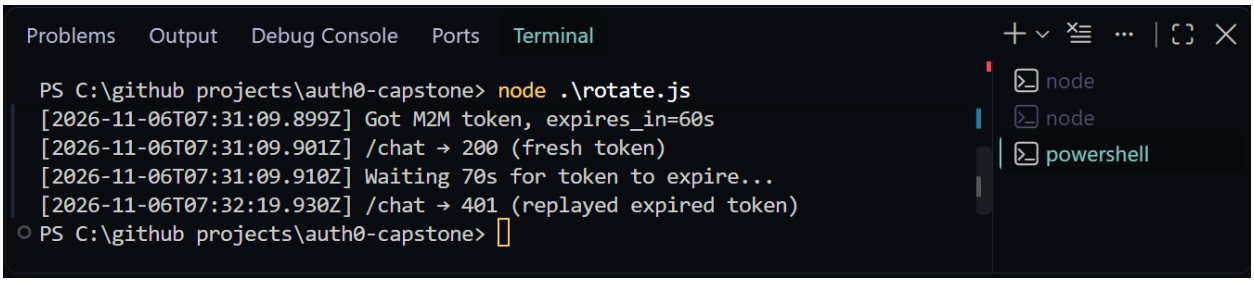


Figure 9: Showing a 200 OK call with the fresh M2M token followed by 401 Unauthorized when the expired token is replayed.

5. Auth0 Actions and SSO

5.1 Post-login Action: agent_id custom claim

I created and deployed a post-login Action named `add-agent-id` and attached it to the Login flow. It adds a custom claim, `agent_id`, to the ID token (and the access token):

Auth0 requires custom claims to use a namespaced (URL-style) name, so the claim appears in the token as `https://ai-chat-api/agent_id`.

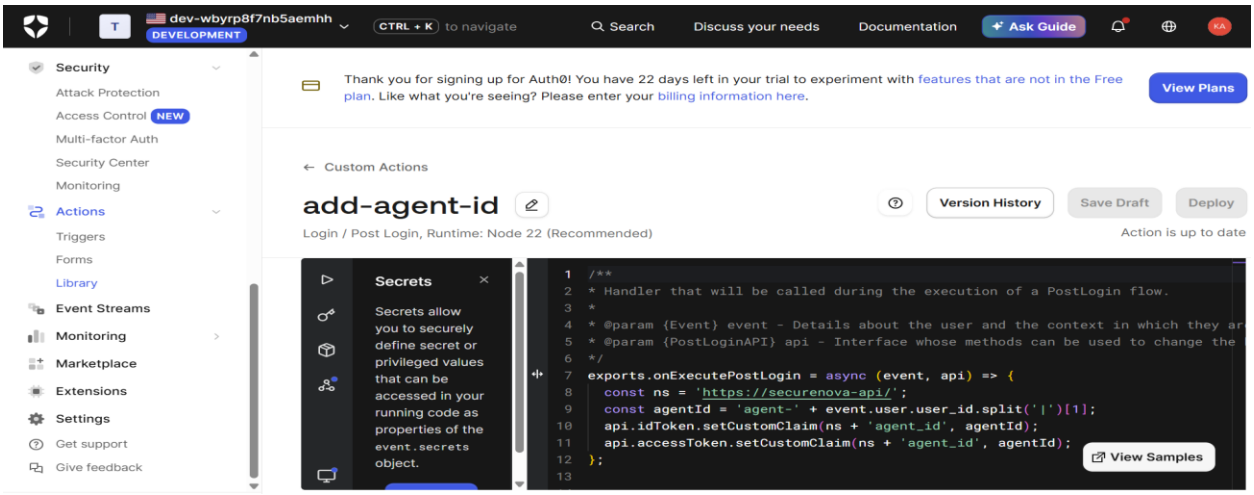


Figure 10: Auth0 Actions editor showing the deployed post-login Action that adds the `agent_id` custom claim.

After deploying the Action and logging in again, I decoded the new token at jwt.io and confirmed that the `agent_id` claim is present.

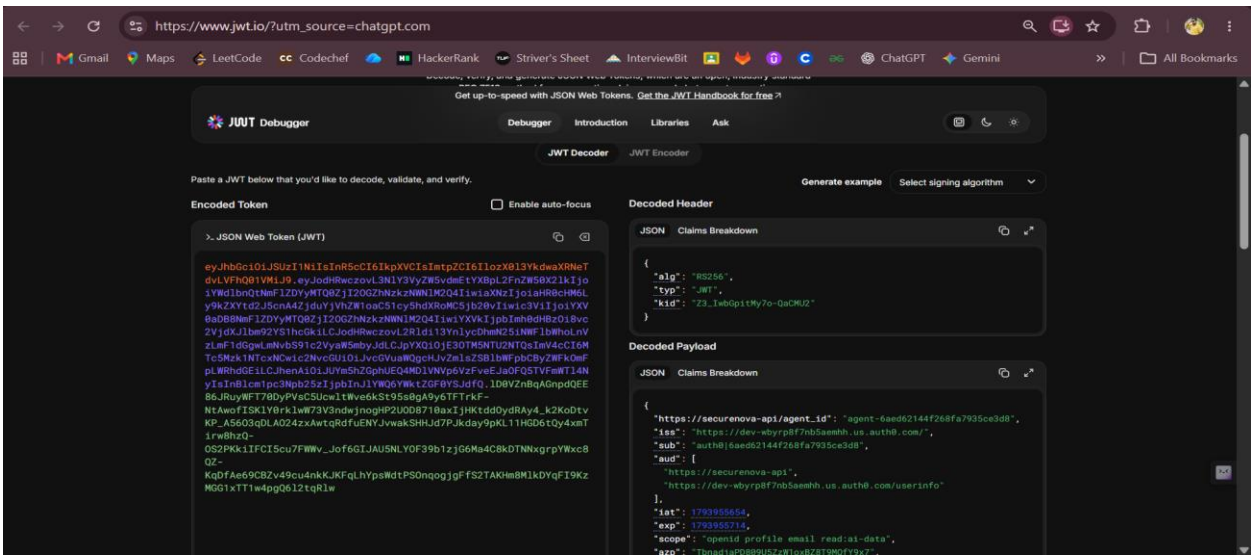


Figure 11: Confirming that the `agent_id` claim is present.

5.2 Single Sign-On (SSO)

I registered a second client application (SecureNova AI Chat 2) in the same tenant, running on port 3001. After authenticating to the first client (port 3000), I opened the second client and requested authorization with `prompt=none`. Auth0 recognized the existing session cookie and returned an authorization code to the second client without showing a login or MFA prompt. This confirms that a user authenticated to one client is accepted by a second registered client without re-authentication.

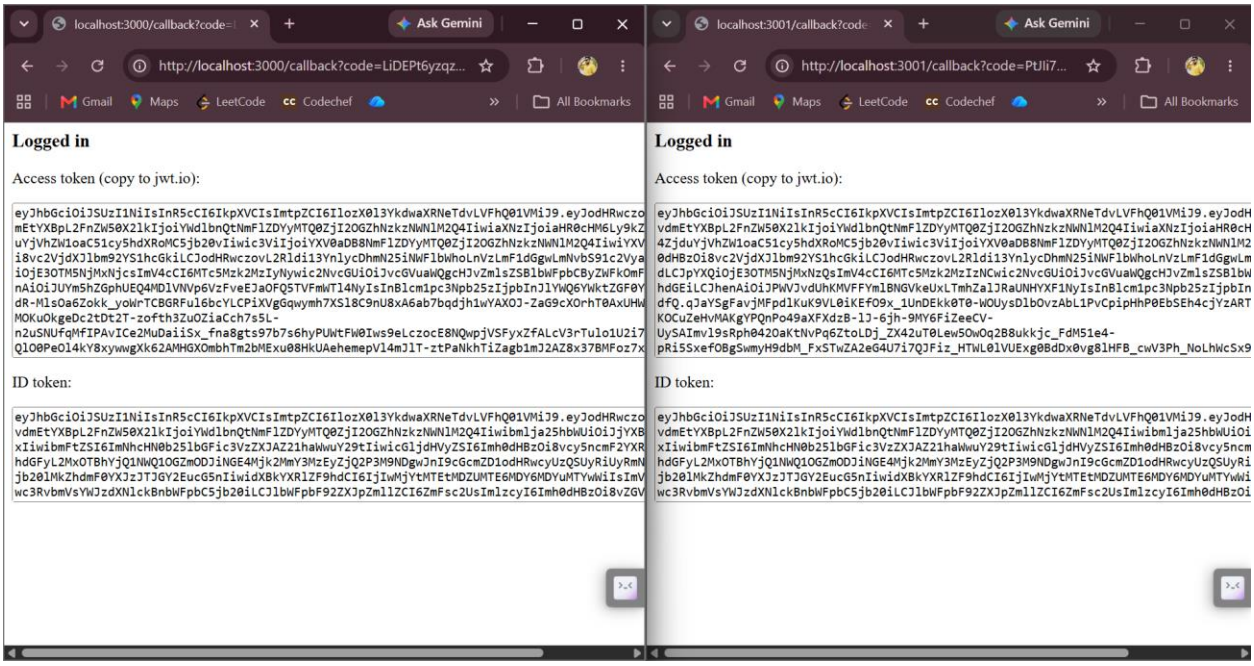


Figure 12: Second client application (port 3001) receiving a successful login via SSO with no re-authentication prompt.

Conclusion

The application uses Authorization Code + PKCE with RS256-signed JWTs, enforces least privilege through the `read:ai-data` and `write:admin` scopes, requires TOTP MFA for all users, and is protected by Brute-Force Protection and Suspicious IP Throttling. M2M credentials are short-lived (60 seconds) and expire as designed. A post-login Action adds an `agent_id` claim, and SSO gives a seamless experience across registered clients.

---------- **END OF PROJECT 2** ----------